MÁSTER EN IA APLICADA · UNIVERSIDAD EUROPEA · PROGRAMACIÓN Y ENTORNOS EN IA

Misión 3 · Los datos tienen que vivir en algún sitio: SQL

Equipo: Yulianis Valero y Yuleimis Valero · **Entorno:** MySQL 8.4 + Workbench · **Repositorio:** github.com/valeroyulianis-commits/mision-3-sql

Organizamos los datos de clientes y operaciones del caso *AI Customer Intelligence* en una base de datos MySQL y preparamos un dataset para un futuro modelo de abandono. Las consultas completas están en **03_retos.sql**.

Reto 1 · Crear la base de datos y revisar sus claves

Creamos la base **ai_customer** con las tablas **clientes** (30 filas) y **operaciones** (99 filas). La relación es **1:N**: un cliente puede tener muchas operaciones, pero cada operación es de un solo cliente. Probamos la clave foránea y MySQL no dejó registrar una compra de un cliente que no existe.

Figura 1.1. Creación de la base y las tablas

```
Ejecución completa de 01_creacion.sql
```

```
Output
#1: OK, 1 row affected in 53.94ms
#2: OK, 0 records retrieved in 0.607ms
#3: OK, 0 records retrieved in 2.814ms
#4: OK, 0 records retrieved in 45.819ms
#5: OK, 0 records retrieved in 46.095ms
#6: OK, 0 records retrieved in 43.415ms
✓ SQL Script execution completed in 478ms. 6 statements executed successfully.
```

Figura 1.2. Carga de 30 clientes y 99 operaciones

```
Ejecución completa de 02_datos.sql
```

```
Output
#1: OK, 0 records retrieved in 0.951ms
#2: OK, 30 rows affected in 54.541ms
#3: OK, 99 rows affected in 52.787ms
✓ SQL Script execution completed in 260ms. 3 statements executed successfully.
```

Figura 1.3. Estructura de clientes (PK id_cliente)

```
DESCRIBE clientes;
```

Output | Result #3

Field	Type	Null	Key	Default	Extra
id_cliente	int	NO	PRI	NULL	
nombre	varchar(80)	NO		NULL	
ciudad	varchar(50)	NO		NULL	
antiguedad_meses	int	NO		NULL	
abandono	tinyint	NO		NULL	

Figura 1.4. Estructura de operaciones (FK id_cliente = MUL)

```
DESCRIBE operaciones;
```

Output | Result #4

Field	Type	Null	Key	Default	Extra
id_operacion	int	NO	PRI	NULL	
id_cliente	int	NO	MUL	NULL	
fecha	date	NO		NULL	
importe	decimal(10,2)	NO		NULL	
canal	varchar(30)	NO		NULL	

Figura 1.5. Claves primarias y foránea

```
SELECT … FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA = 'ai_customer';
```

Output | Result #5 | Result #6

TABLE_NAME	CONSTRAINT_NAME	COLUMN_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
clientes	PRIMARY	id_cliente	NULL	NULL
operaciones	fk_operaciones_cliente	id_cliente	clientes	id_cliente
operaciones	PRIMARY	id_operacion	NULL	NULL

Figura 1.6. Recuento de filas

```
SELECT COUNT(*) FROM clientes UNION ALL SELECT COUNT(*) FROM operaciones;
```

Output | Result #5 | Result #6

tabla	filas
clientes	30
operaciones	99

Figura 1.7. Prueba de la FK: Error 1452 al insertar un cliente inexistente

```
INSERT INTO operaciones VALUES (999, 999, '2026-09-01', 50.00, 'web');
```

Output

```
#1: OK, 0 records retrieved in 0.823ms
#2: MySQL Error (1452): Cannot add or update a child row: a foreign key constraint fails (`ai_customer`.`operaciones`, CONSTRAINT `fk_operaciones_cliente
✕ SQL Script execution completed in 169ms. 2 statements executed, 1 error occurred.
```

Reto 2 · Filtros con WHERE y ORDER BY

Filtramos los clientes de Madrid (5), los de menos de 6 meses (5) y los que abandonaron (9), y ordenamos por antigüedad. **WHERE** elige qué filas salen; **ORDER BY** solo cambia el orden.

Figura 2.1. Clientes de Madrid

```
SELECT * FROM clientes WHERE ciudad = 'Madrid';
```

Output | Result #8

id_cliente	nombre	ciudad	antiguedad_meses	abandono
1	Cliente 01	Madrid	1	1
7	Cliente 07	Madrid	3	1
13	Cliente 13	Madrid	9	0
19	Cliente 19	Madrid	9	0
25	Cliente 25	Madrid	27	0

Figura 2.2. Antigüedad menor a 6 meses

```
SELECT * FROM clientes WHERE antiguedad_meses < 6;
```

Output | Result #9

id_cliente	nombre	ciudad	antiguedad_meses	abandono
1	Cliente 01	Madrid	1	1
7	Cliente 07	Madrid	3	1
16	Cliente 16	Sevilla	3	0
18	Cliente 18	Zaragoza	4	0
30	Cliente 30	Zaragoza	2	0

Figura 2.3. Orden de mayor a menor antigüedad (primeras filas)

```
SELECT … FROM clientes ORDER BY antiguedad_meses DESC;
```

Output | Result #10

id_cliente	nombre	ciudad	antiguedad_meses
15	Cliente 15	Valencia	48
22	Cliente 22	Sevilla	47
12	Cliente 12	Zaragoza	46
6	Cliente 06	Zaragoza	44
17	Cliente 17	Bilbao	43
27	Cliente 27	Valencia	38
10	Cliente 10	Sevilla	35
21	Cliente 21	Valencia	35
24	Cliente 24	Zaragoza	34
2	Cliente 02	Barcelona	31

Figura 2.4. Clientes con abandono = 1

```
SELECT * FROM clientes WHERE abandono = 1;
```

Output | Result #11

id_cliente	nombre	ciudad	antiguedad_meses	abandono
1	Cliente 01	Madrid	1	1
7	Cliente 07	Madrid	3	1
8	Cliente 08	Barcelona	9	1
14	Cliente 14	Barcelona	10	1
15	Cliente 15	Valencia	48	1
21	Cliente 21	Valencia	35	1
22	Cliente 22	Sevilla	47	1
28	Cliente 28	Sevilla	6	1
29	Cliente 29	Bilbao	8	1

Reto 3 · Combinar tablas con JOIN

Unimos operaciones con el nombre de su cliente (INNER JOIN, 99 filas) y encontramos 4 clientes sin compras con LEFT JOIN. También filtramos compras mayores a 200 € (37) y las del Cliente 03 (6). **INNER JOIN** muestra solo lo que coincide en ambas tablas; **LEFT JOIN** muestra todos los clientes aunque no tengan compras.

Figura 3.1. Operaciones con nombre del cliente (primeras filas)

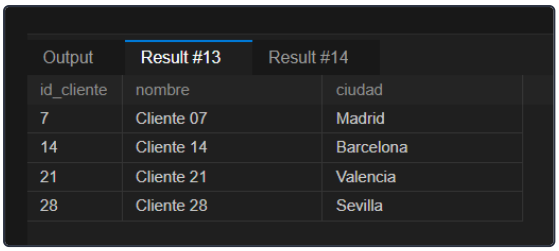
```
FROM operaciones o INNER JOIN clientes c ON o.id_cliente = c.id_cliente
```

Output | Result #12

id_operacion	fecha	importe	canal	id_cliente	nombre
1	07/06/2026	79.87	app	1	Cliente 01
2	20/06/2026	315.86	web	1	Cliente 01
3	14/04/2026	199.24	tienda	1	Cliente 01
4	02/01/2026	279.19	app	1	Cliente 01
5	05/04/2026	278.47	app	2	Cliente 02
6	03/01/2026	246.54	app	2	Cliente 02

Figura 3.2. Clientes sin operaciones

```
FROM clientes c LEFT JOIN operaciones o ON … WHERE o.id_operacion IS NULL
```

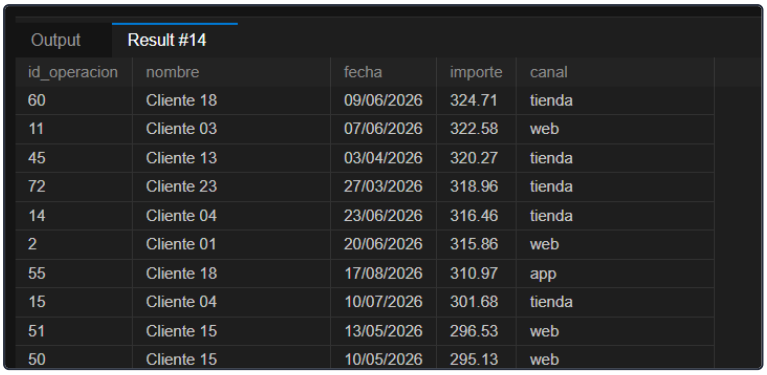


Output | Result #13 | Result #14

id_cliente	nombre	ciudad
7	Cliente 07	Madrid
14	Cliente 14	Barcelona
21	Cliente 21	Valencia
28	Cliente 28	Sevilla

Figura 3.3. Operaciones mayores a 200 € (primeras filas)

```
… INNER JOIN clientes c ON … WHERE o.importe > 200 ORDER BY o.importe DESC
```

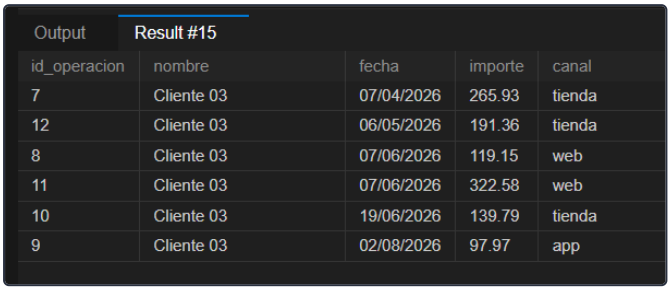


Output | Result #14

id_operacion	nombre	fecha	importe	canal
60	Cliente 18	09/06/2026	324.71	tienda
11	Cliente 03	07/06/2026	322.58	web
45	Cliente 13	03/04/2026	320.27	tienda
72	Cliente 23	27/03/2026	318.96	tienda
14	Cliente 04	23/06/2026	316.46	tienda
2	Cliente 01	20/06/2026	315.86	web
55	Cliente 18	17/08/2026	310.97	app
15	Cliente 04	10/07/2026	301.68	tienda
51	Cliente 15	13/05/2026	296.53	web
50	Cliente 15	10/05/2026	295.13	web

Figura 3.4. Operaciones del Cliente 03

```
… INNER JOIN clientes c ON … WHERE c.id_cliente = 3
```



Output | Result #15

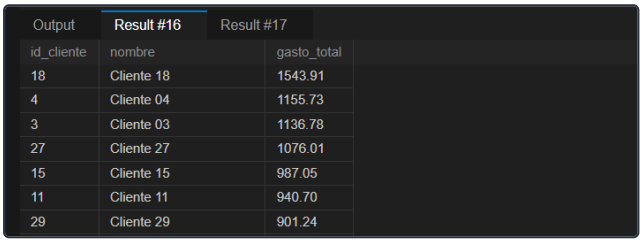
id_operacion	nombre	fecha	importe	canal
7	Cliente 03	07/04/2026	265.93	tienda
12	Cliente 03	06/05/2026	191.36	tienda
8	Cliente 03	07/06/2026	119.15	web
11	Cliente 03	07/06/2026	322.58	web
10	Cliente 03	19/06/2026	139.79	tienda
9	Cliente 03	02/08/2026	97.97	app

Reto 4 · Calcular indicadores

Calculamos el gasto total por cliente, las operaciones por canal, el ticket medio (172,97 €) y los 16 clientes que gastaron más de 500 €. **WHERE** filtra filas antes de agrupar; **HAVING** filtra los grupos después, por eso puede usar SUM.

Figura 4.1. Gasto total por cliente (primeras filas)

```
SELECT c.id_cliente, SUM(o.importe) … GROUP BY c.id_cliente, c.nombre
```



Output | Result #16 | Result #17

id_cliente	nombre	gasto_total
18	Cliente 18	1543.91
4	Cliente 04	1155.73
3	Cliente 03	1136.78
27	Cliente 27	1076.01
15	Cliente 15	987.05
11	Cliente 11	940.70
29	Cliente 29	901.24

Figura 4.2. Operaciones por canal

```
SELECT canal, COUNT(*), SUM(importe) FROM operaciones GROUP BY canal
```

Output | Result #16 | **Result #17**

canal	num_operaciones	importe_total
app	38	6090.02
tienda	36	6442.13
web	25	4591.71

Figura 4.3. Ticket medio global

```
SELECT ROUND(AVG(importe), 2) FROM operaciones;
```

Output | Result #17 | **Result #18**

ticket_medio_global
172.97

Figura 4.4. Ticket medio por cliente (primeras filas)

```
SELECT c.id_cliente, ROUND(AVG(o.importe), 2) … GROUP BY c.id_cliente, c.nombre
```

Output | **Result #19** | Result #20

id_cliente	nombre	ticket_medio
13	Cliente 13	320.27
26	Cliente 26	262.56
2	Cliente 02	262.51
18	Cliente 18	257.32
19	Cliente 19	244.78
1	Cliente 01	218.54
23	Cliente 23	207.97
16	Cliente 16	205.07
4	Cliente 04	192.62

Figura 4.5. Clientes con gasto > 500 € (HAVING)

```
… GROUP BY c.id_cliente, c.nombre HAVING SUM(o.importe) > 500
```

Output | **Result #20**

id_cliente	nombre	gasto_total
18	Cliente 18	1543.91
4	Cliente 04	1155.73
3	Cliente 03	1136.78
27	Cliente 27	1076.01
15	Cliente 15	987.05
11	Cliente 11	940.70
29	Cliente 29	901.24
10	Cliente 10	886.69
1	Cliente 01	874.16
23	Cliente 23	831.87
9	Cliente 09	808.12
30	Cliente 30	786.78
22	Cliente 22	660.29
24	Cliente 24	618.75
8	Cliente 08	586.81
2	Cliente 02	525.01

Figura 4.6. WHERE + HAVING: más de 500 € solo en la app

```
… WHERE canal = 'app' GROUP BY id_cliente HAVING SUM(importe) > 500
```

```
144  SELECT id_cliente, SUM(importe) AS gasto_app
145  FROM operaciones
146  WHERE canal = 'app'
147  GROUP BY id_cliente
148  HAVING SUM(importe) > 500;
149
```

Output | Result #21

id_cliente	gasto_app
2	525.01
8	536.52
18	887.36
23	512.91

Reto final · Dataset para IA

Generamos una fila por cliente con id_cliente, antiguedad_meses, num_operaciones, gasto_total, ticket_medio y abandono. Con **LEFT JOIN** incluimos a los clientes sin compras y con **COALESCE** sus valores salen en 0 en vez de NULL. Validamos: **30 filas, sin duplicados y sin nulos**.

Figura F.1. Dataset final (04_dataset_ia.sql)

```
COUNT(o.id_operacion), COALESCE(SUM(o.importe),0), COALESCE(AVG(o.importe),0) … FROM clientes c LEFT JOIN operaciones o … GROUP BY c.id_cliente
```

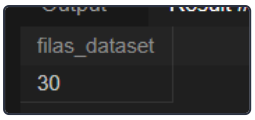
```
1   USE ai_customer;
2   SELECT c.id_cliente, c.antiguedad_meses,
3          COUNT(o.id_operacion) AS num_operaciones,
4          COALESCE(SUM(o.importe),0) AS gasto_total,
5          COALESCE(AVG(o.importe),0) AS ticket_medio,
6          c.abandono
7   FROM clientes c
8   LEFT JOIN operaciones o ON c.id_cliente=o.id_cliente
9   GROUP BY c.id_cliente,c.antiguedad_meses,c.abandono
10  ORDER BY c.id_cliente;
11
```

Output | Result #2

id_cliente	antiguedad_meses	num_operaciones	gasto_total	ticket_medio	abandono
1	1	4	874.16	218.540000	1
2	31	2	525.01	262.505000	0
3	8	6	1136.78	189.463333	0
4	26	6	1155.73	192.621667	0
5	10	1	120.59	120.590000	0
6	44	2	304.23	152.115000	0
7	3	0	0.00	0.000000	1
8	9	4	586.81	146.702500	1
9	8	5	808.12	161.624000	0
10	35	5	886.69	177.338000	0
11	15	6	940.70	156.783333	0
12	46	3	211.58	70.526667	0

Figura F.2. Validación: 30 filas

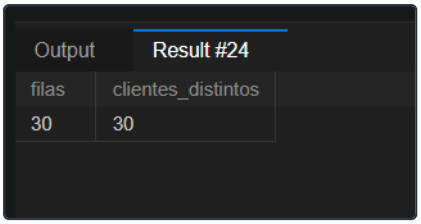
```
SELECT COUNT(*) FROM (dataset) AS d;
```



filas_dataset
30

Figura F.3. Validación: sin duplicados

```
SELECT COUNT(*), COUNT(DISTINCT id_cliente) FROM (dataset) AS d;
```

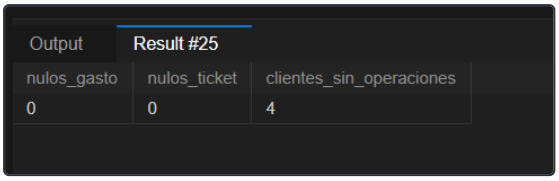


Output | Result #24

filas	clientes_distintos
30	30

Figura F.4. Validación: sin nulos

```
SELECT SUM(gasto_total IS NULL), SUM(ticket_medio IS NULL) FROM (dataset) AS d;
```



Output | Result #25

nulos_gasto	nulos_ticket	clientes_sin_operaciones
0	0	4

Entrega en GitHub

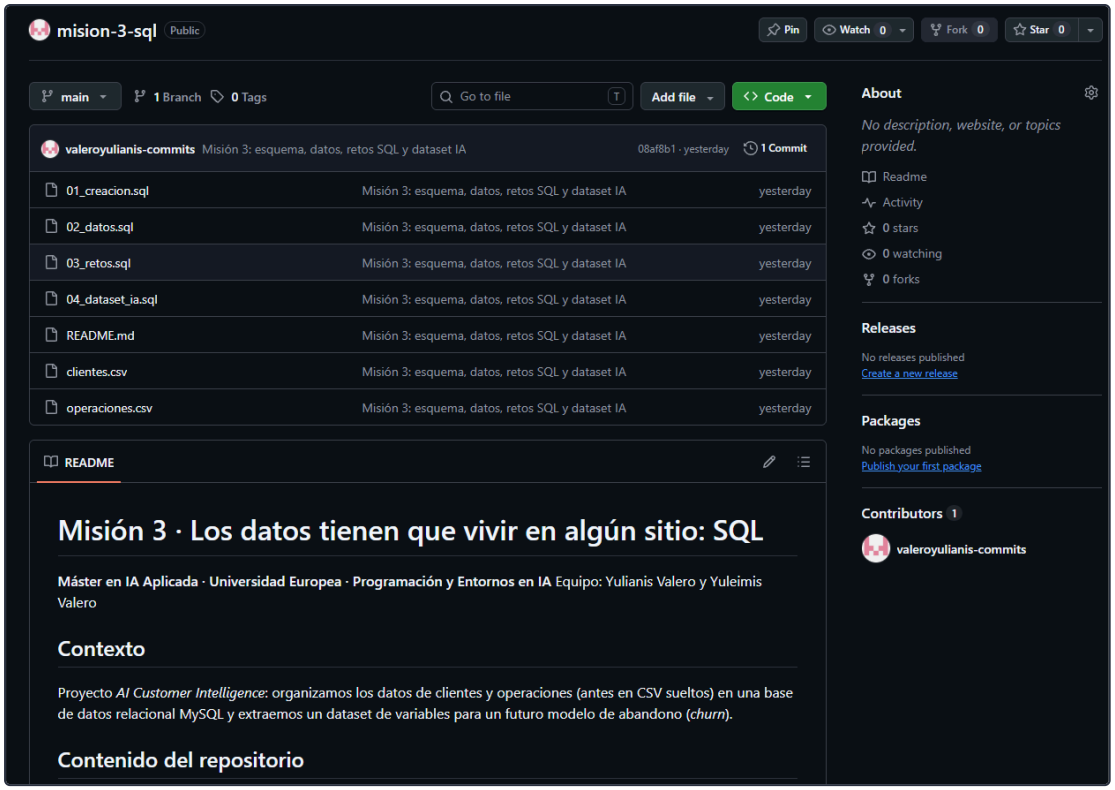
> Subimos los scripts, los CSV y el README al repositorio **mision-3-sql**.

Figura G.1. Push a GitHub

```
git add … · git commit -m "Misión 3: …" · git push -u origin main
```

```
PS C:\Users\Yulianis Valero\Downloads\Mision_3_SQL_material_completo Alumnos\Mision_3_SQL_material_completo Alumnos> git
 push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 6.78 KiB | 3.39 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/valeroyulianis-commits/mision-3-sql.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Yulianis Valero\Downloads\Mision_3_SQL_material_completo Alumnos\Mision_3_SQL_material_completo Alumnos>
```

Figura G.2. Repositorio mision-3-sql



> **Conclusión:** los clientes que abandonan compran menos (445 € de media frente a 624 € de los que se quedan) y los 4 que nunca compraron abandonaron todos. El dataset queda listo para cargarlo en Python en la Misión 4.

Uso de IA: usamos IA generativa como asistente para escribir y depurar las consultas de 03_retos.sql; las ejecutamos y validamos nosotras en MySQL y entendemos lo que hace cada una.